Сортировки

Сортировка пузырьком, сортировка вставками, сортировка корзинами (bucket sort), сортировка подсчётом (counting sort), поразрядная сортировка (radix sort), пирамидальная сортировка (heap sort)

Простые сортировки с прямым сравнением элементов

Сортировка пузырьком

Одна из наиболее простых сортировок построена на том, что мы проверяем каждую пару элементов на то, что она следует в правильном порядке (элемент с меньшим индексом имеет меньшее значение), иначе меняем значения местами. Если в какой-то момент оказалось, что мы не сделали ни одной перестановки, значит массив отсортирован.

```
void bubble_sort(int arr[], unsigned const begin, unsigned const end) {
    bool sorted = false;
    while (!sorted) {
        sorted = true;
        for (unsigned idx = begin; idx != end - 1; ++idx)
            if (arr[idx] > arr[idx + 1]) {
                auto tmp = arr[idx];
                arr[idx] = arr[idx + 1];
                arr[idx + 1] = tmp;
                sorted = false;
            }
    }
}
```



Проходим по массиву и проверяем верный порядок элементов.

Сортировка пузырьком

Сортировка пузырьком так устроена, что наибольший (или наименьший) элемент словно «всплывает» через массив к краю. Таким образом каждый проход максимальный элемент занимает верное положение. Опираясь на этот факт, мы можем переписать сортировку.

```
3 1 12 7 10 8 6 5 9 11 13 15 2 4 14 0
1 3 7 10 8 6 5 9 11 12 13 2 4 14 0 15
1 3 7 8 6 5 9 10 11 12 2 4 13 0 14 15
1 3 7 6 5 8 9 10 11 2 4 12 0 13 14 15
1 3 6 5 7 8 9 10 2 4 11 0 12 13 14 15
1 3 5 6 7 8 9 2 4 10 0 11 12 13 14 15
1 3 5 6 7 8 2 4 9 0 10 11 12 13 14 15
1 3 5 6 7 2 4 8 0 9 10 11 12 13 14 15
1 3 5 6 2 4 7 0 8 9 10 11 12 13 14 15
1 3 5 2 4 6 0 7 8 9 10 11 12 13 14 15
1 3 2 4 5 0 6 7 8 9 10 11 12 13 14 15
1 2 3 4 0 5 6 7 8 9 10 11 12 13 14 15
1 2 3 0 4 5 6 7 8 9 10 11 12 13 14 15
1 2 0 3 4 5 6 7 8 9 10 11 12 13 14 15
1 0 2 3 4 5 6 7 8 9 10 11 12 13 14 15
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

```cpp
bool sorted = false;
for (unsigned last = end - 1; last != 0 and !sorted; --last) {
    sorted = true;
    for (unsigned idx = begin; idx != last; ++idx)
        if (arr[idx] > arr[idx + 1]) {
            auto tmp = arr[idx];
            arr[idx] = arr[idx + 1];
            arr[idx + 1] = tmp;
            sorted = false;
        }
}
```

В наихудшем случае этот алгоритм имеет асимптотику $O(N^2)$ и в наилучшем – $O(N)$.
В среднем случае сортировка работает за $O(N^2)$.

Сортировка вставками

Идея сортировки вставками состоит в том, что массив делится на две части: отсортированную и не отсортированную. Каждый новый элемент из неотсортированной части продвигается в отсортированную часть до тех пор, пока не «упирается» в элемент больше него.

```
void insert_sort(int arr[], unsigned const begin, unsigned const end) {
    for (unsigned sorted = end - 1; sorted != 0; --sorted)
        for (unsigned idx = sorted - 1; idx != end - 1; ++idx) {
            if (arr[idx] <= arr[idx + 1])
                break;
            auto tmp = arr[idx];
            arr[idx] = arr[idx + 1];
            arr[idx + 1] = tmp;
        }
}
```

В лучшем случае сортировка имеет асимптотическую сложность O(N).
В худшем случае сортировка имеет асимптотическую сложность $O(N^2)$.
В среднем случае сортировка имеет асимптотическую сложность $O(N^2)$.

Сортировки значений из небольших множеств

когда мощность множества значений много меньше количества элементов

Сортировка булевых значений

Если мы сортируем булевы значения, то мы можем опираться на тот факт, что есть всего два возможных значения: true и false. Это позволяет нам сразу располагать элементы на тех местах, где они должны находится, не сравнивая между собой – ложь в начале массива и истину в конце массива.

```
void bool_sort(bool arr[], bool tmp[], unsigned const begin, unsigned const end) {
    unsigned w_idx = begin;
    for (unsigned idx = begin; idx != end; ++idx)
        if (!arr[idx])
            tmp[w_idx++] = arr[idx];

    for (unsigned idx = begin; idx != end; ++idx)
        if (arr[idx])
            tmp[w_idx++] = arr[idx];

    for (unsigned idx = begin; idx != end; ++idx)
        arr[idx] = tmp[idx];
}
```

записываем значения false

записываем значения true

копируем из временной памяти в основной массив

Этот алгоритм работает с линейной асимптотикой по времени O(N), но требует от нас дополнительной памяти для промежуточного сохранения всех элементов, т.е. порядка N. *Говорят, что алгоритм использует O(N) дополнительной памяти*.

```
true false true true false true false false false false
false false false false false false true true true true
```

Сортировка булевых значений

Если сами элементы не важны, а важны только их значения, и мы можем свободно их создавать и копировать (это не всегда так, но верно для булевых значений), мы можем просто подсчитать сколько у нас есть элементов с заданными значениями и создать их заново в исходном массиве.

```
void bool_sort(bool arr[], unsigned const begin, unsigned const end) {
    unsigned false_count = 0;
    for (unsigned idx = begin; idx != end; ++idx)
        if (!arr[idx]) ++false_count;

    unsigned w_idx = begin;
    for (; false_count != 0; --false_count, ++w_idx)
        arr[w_idx] = false; записываем новое значение false
    for (; w_idx != end; ++w_idx)
        arr[w_idx] = true;  записываем новое значение true
}
```

Эта сортировка также имеет линейную асимптотику по количеству данных, но не использует дополнительной памяти. Однако массив заполняется новыми значениями, а теми, что уже содержались в массиве.

Сортировка корзинами

Пусть у нас есть массив целых чисел, но в нём встречаются только значения от 0 до 9. Тогда разбросаем элементы по корзинам, а затем скопируем элементы обратно.

```
void bucket_sort(int arr[], int buckets[][10], unsigned const begin, unsigned const end) {
    unsigned buckets_count[10] = {0};
    for (unsigned idx = begin; idx != end; ++idx)
        buckets[ buckets_count[arr[idx]]++ ][ arr[idx] ] = arr[idx];

    unsigned w_idx = 0;
    for (unsigned b_idx = 0; b_idx != 10; ++b_idx)
        for (unsigned idx = 0; idx != buckets_count[b_idx]; ++idx)
            arr[w_idx++] = buckets[idx][b_idx];
}
```

```
1 0 2 2 3 5 9 8 6 6 1 0 3 5 5 5 7 8 9 0
3 2 2 2 0 4 2 1 2 2
0 0 0 1 1 2 2 3 3 5 5 5 5 6 6 7 8 8 9 9
```

Исходные 20 значений

Количество элементов по корзинам

Отсортированный массив

Сортировка подсчётом

Пусть у нас есть массив целых чисел, но в нём встречаются только значения от 0 до 9. Тогда подсчитаем сколько раз каждое значение встречается в массиве и затем столько раз запишем его в исходный массив.

```
void count_sort(int arr[], unsigned const begin, unsigned const end) {
    unsigned buckets_count[10] = {0};
    for (unsigned idx = begin; idx != end; ++idx)
        ++buckets_count[arr[idx]];

    unsigned w_idx = 0;
    for (unsigned b_idx = 0; b_idx != 10; ++b_idx)
        for (unsigned idx = 0; idx != buckets_count[b_idx]; ++idx)
            arr[w_idx++] = b_idx;
}
```

Сортировка использует O(1) дополнительной памяти, но заменяет элементы массива новыми. Асимптотическая сложность O(N).

Поразрядная сортировка

сортировка составных значений на основе сортировке корзинами

Пример поразрядной сортировки чисел

Расставим элементы таким образом, чтобы те элементы, у которых младший разряд меньше, шли раньше в массиве, но с сохранением относительного порядка. Затем выполним ту же операцию, но для следующего разряда. И т.д. Иначе: по каждому разряду мы производим сортировку корзинами.

1 3 4 5 6	0 2 5 7 9	1 2 5 8 1	1 2 5 8 1	2 5 4 6 2	4 4 3 0 5
0 2 1 0 3	1 3 4 5 6	0 2 5 7 9	0 2 5 7 9	4 4 3 0 5	3 2 4 5 6
3 2 4 5 6	3 2 4 5 6	3 1 5 7 4	3 1 5 7 4	1 3 4 5 6	3 1 5 7 4
1 2 5 8 1	0 0 0 4 5	2 5 4 6 2	2 5 4 6 2	1 2 5 8 1	2 5 4 6 2
0 2 5 7 9	4 4 3 0 5	1 3 4 5 6	1 3 4 5 6	0 2 5 7 9	1 3 4 5 6
0 0 0 4 5	3 1 5 7 4	3 2 4 5 6	3 2 4 5 6	3 2 4 5 6	1 2 5 8 1
2 5 4 6 2	0 2 1 0 3	0 0 0 4 5	4 4 3 0 5	0 2 1 0 3	0 2 5 7 9
0 0 1 2 3	0 0 1 2 3	0 0 1 2 3	0 0 1 2 3	3 1 5 7 4	0 2 1 0 3
4 4 3 0 5	0 0 0 2 3	0 0 0 2 3	0 2 1 0 3	0 0 1 2 3	0 0 1 2 3
0 0 0 2 3	2 5 4 6 2	4 4 3 0 5	0 0 0 4 5	0 0 0 4 5	0 0 0 4 5
3 1 5 7 4	1 2 5 8 1	0 2 1 0 3	0 0 0 2 3	0 0 0 2 3	0 0 0 2 3

Поразрядная сортировка

В случае числовых данных C++ уместнее выбрать двоичные разряды. Тогда поразрядная сортировка будет выглядеть следующим образом:

```
void radix_sort(
    unsigned arr[], unsigned tmp[],
    unsigned const begin, unsigned const end
) {
    for (unsigned shift = 0; shift != 32; ++shift) {
        auto mask = 1u << shift;
        unsigned w_idx = begin;
        for (unsigned r_idx = begin; r_idx != end; ++r_idx)
            if (0 == (mask & arr[r_idx]))
                tmp[w_idx++] = arr[r_idx];
        for (unsigned r_idx = begin; r_idx != end; ++r_idx)
            if (0 != (mask & arr[r_idx]))
                tmp[w_idx++] = arr[r_idx];
        for (unsigned idx = begin; idx != end; ++idx)
            arr[idx] = tmp[idx];
    }

}
```

Так же, как в сортировке булевых значений, сначала мы записываем во временный массив данные с нулевым значением разряда, затем данные с единичным значением разряда.

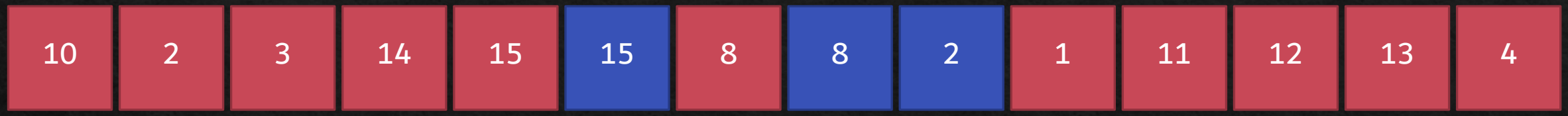
Перезаписываем данные в исходный массив

Поразрядная сортировка

Асимптотическая сложность поразрядной сортировки зависит от максимальной разрядности элемента K - $O(K \cdot N)$. При фиксированной разрядности поразрядная сортировка имеет линейную асимптотическую сложность. Также нам понадобиться дополнительная память для сохранения элементов в соответствующие разрядам корзины.

Понятие устойчивости сортировки

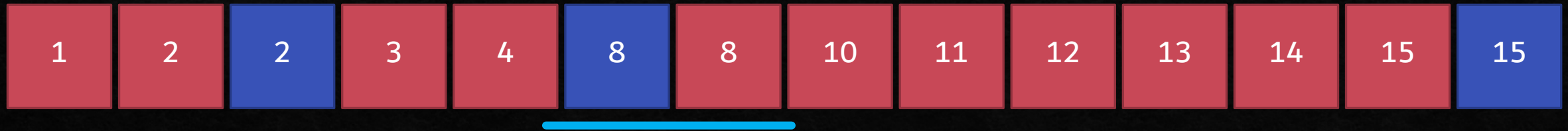
Сортировка называется «устойчивой», если порядок равных элементов в отсортированной последовательности такой же, как и в изначальной последовательности.



Устойчиво отсортированный массив



Неустойчиво отсортированный массив

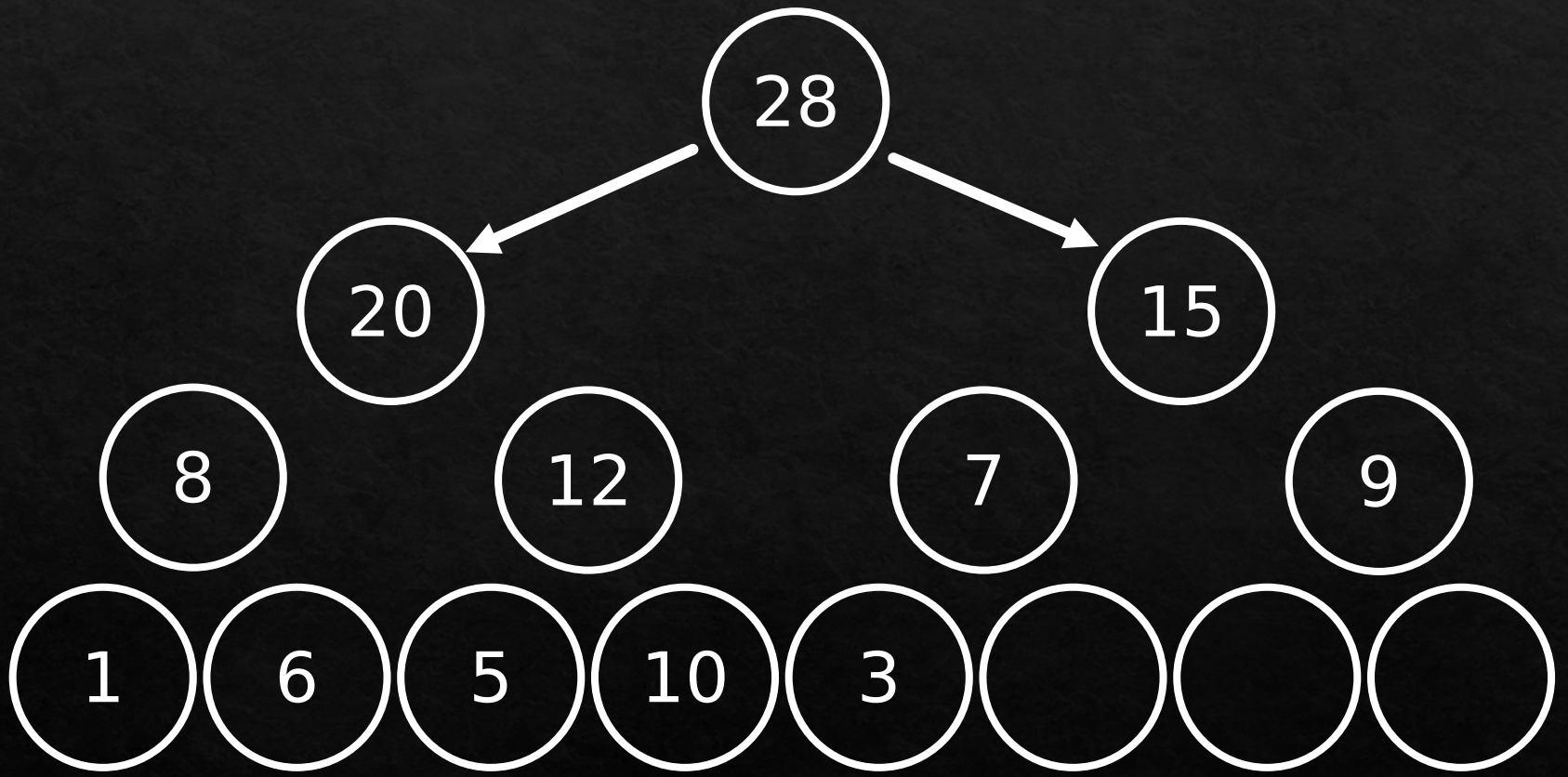


Двоичная куча

эффективное хранение данных для поиска максимального (минимального) значения

Двоичная куча (максимальная)

Расположим элементы в виде пирамиды таким образом, чтобы каждый элемент опирался на два элемента меньше его.

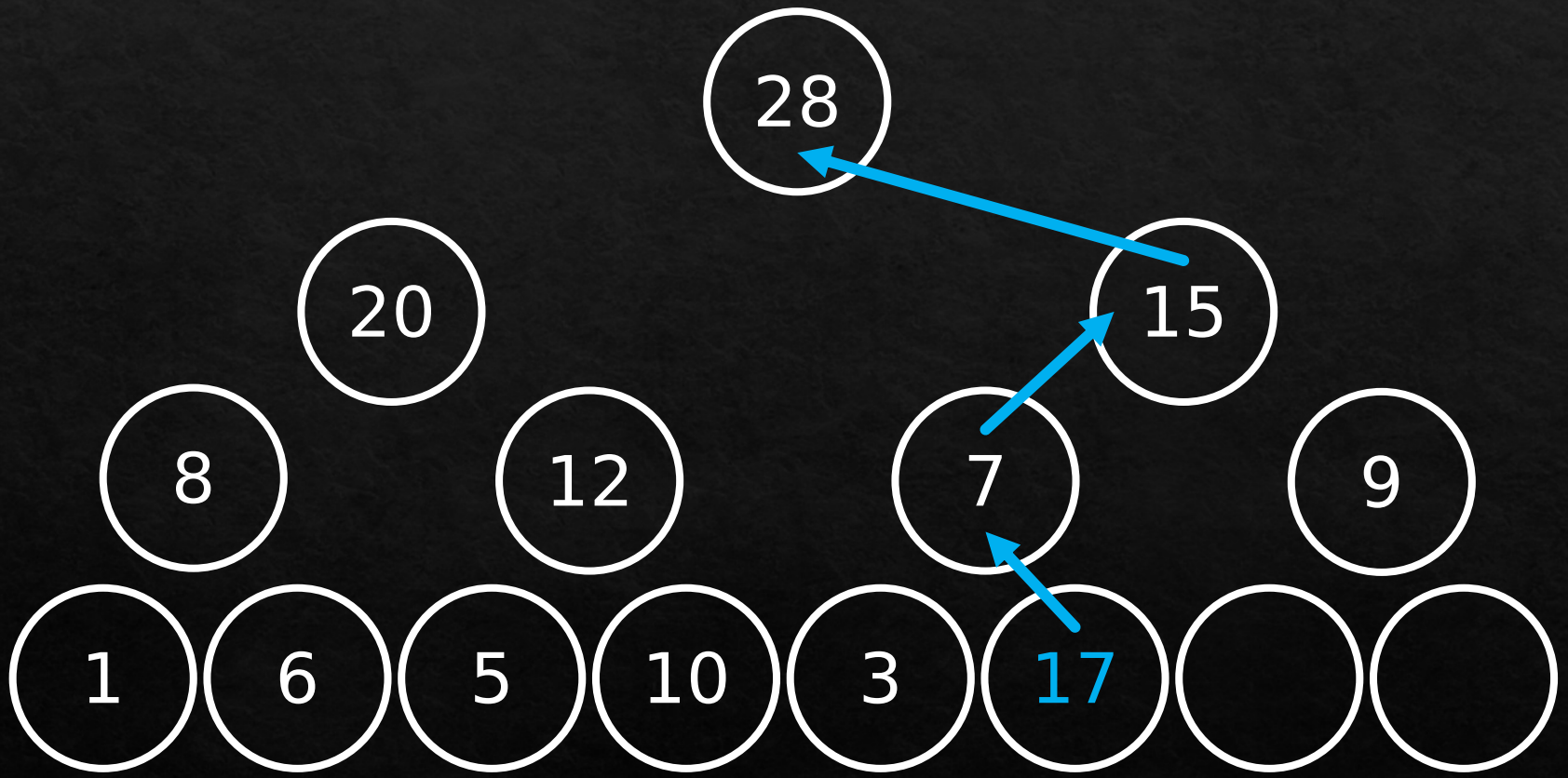


Максимальный элемент расположен на вершине.

В пирамиде могут быть свободные места, но количество элементов на каждом уровне – степень двойки.

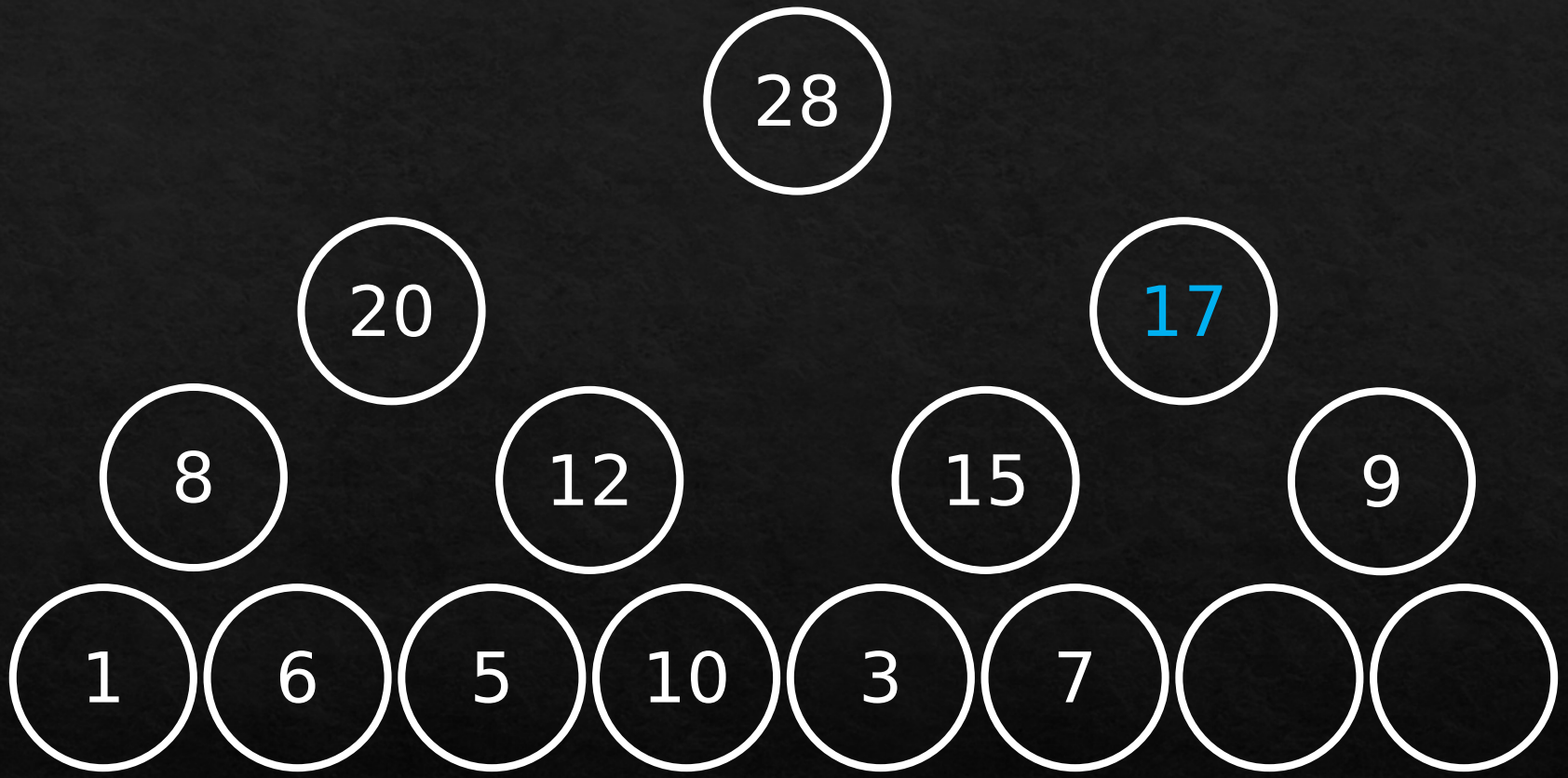
Вставка и просеивание вверх

Новые элементы добавляются в пирамиду начиная со свободных мест. Если мест нет, то нужно организовать новый уровень. Новый элемент просеивается вверх.



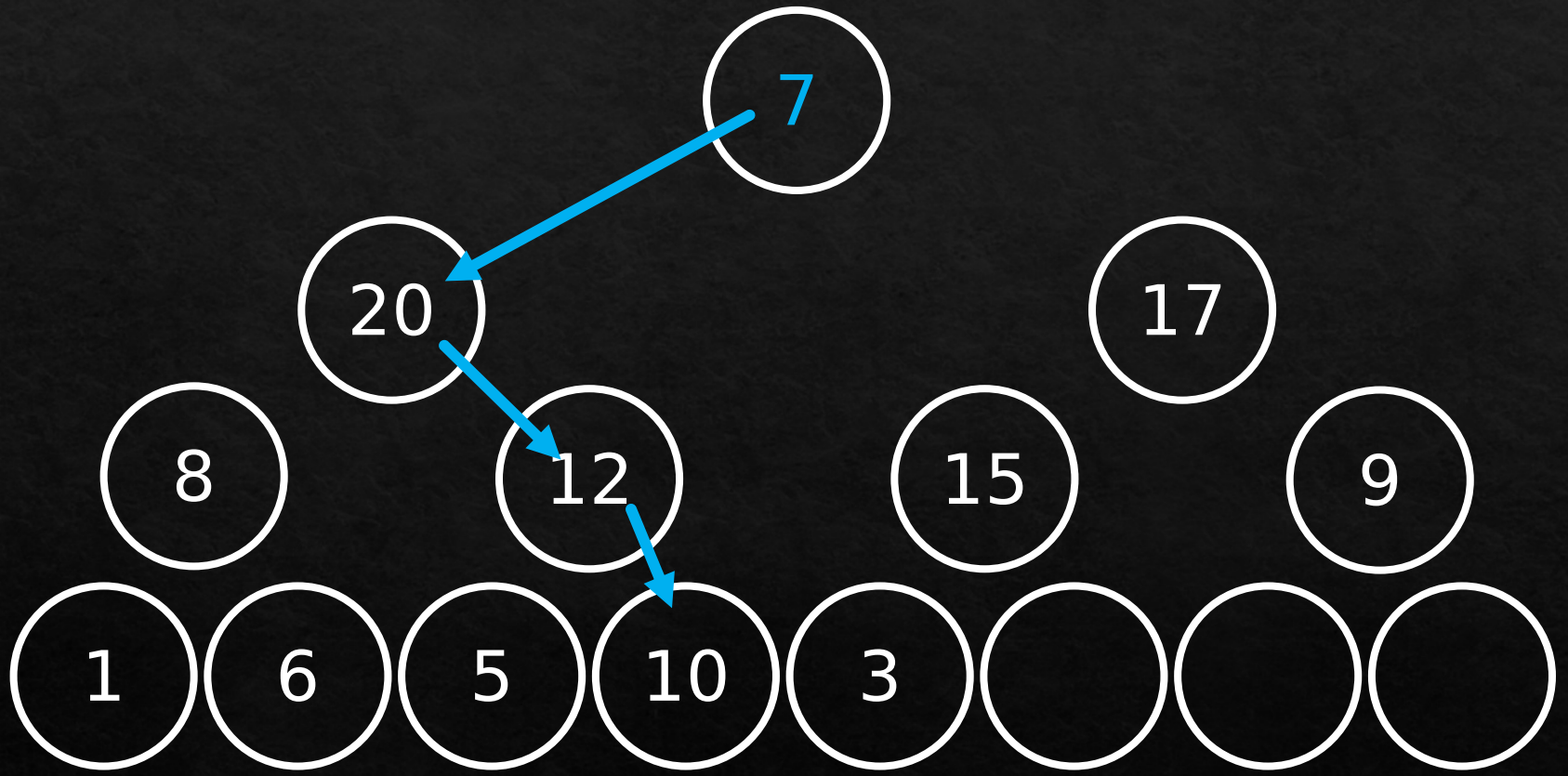
При просеивании мы проверяем, что элемент не превосходит своего родителя. Если это не так, то меняем его местами с родителем и снова повторяем проверку. Проверка заканчивается, когда верх пирамиды достигнут или элемент меньше родительского.

Вставка и просеивание вверх



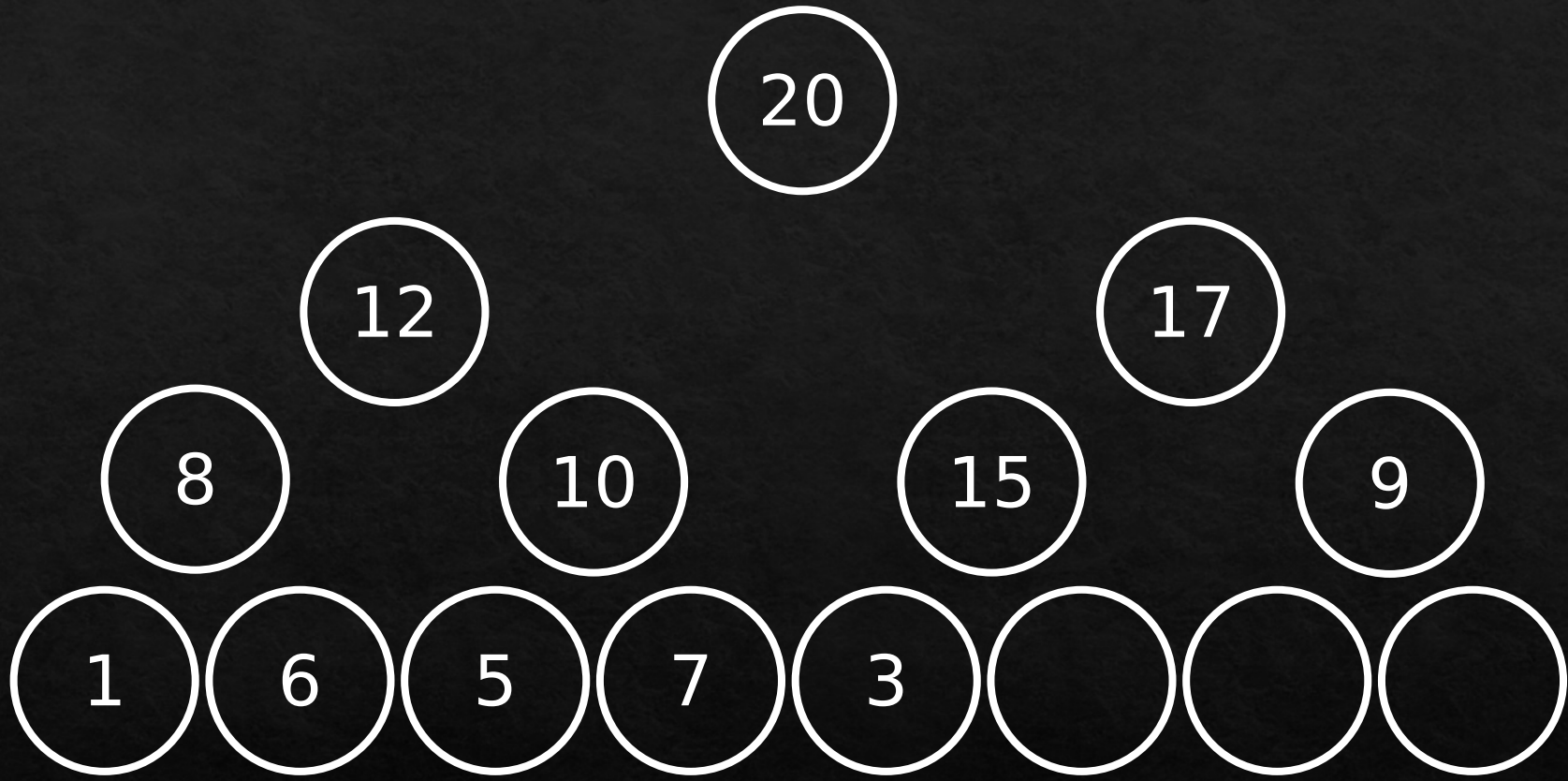
Удаление максимального и просеивание вниз

При удалении максимального элемента мы заменяем его на последний элемент и просеиваем этот элемент вниз.



При просеивании вниз мы смотрим, чтобы наш элемент был больше, чем его потомки. Если это не так, то находим бОльшего потомка и меняем наш элемент с ним местами.

Удаление максимального и просеивание вниз



Отметим, что после этой процедуры на вершине пирамиды вновь оказывается максимальный элемент.

Асимптотическая сложность

Оценим асимптотическую сложность операций.

Самая быстрая операция – узнать максимальное значение – она не зависит от количества элементов O(1).

Добавление нового элемента в пирамиду занимает O(h) действий, где h – высота пирамиды, при условии, что обмен значениями, поиск дочерних элементов и добавление нового свободного места под элемент занимают O(1).

Удаление вершины пирамиды занимает O(h) действий, где h – высота пирамиды при тех же условиях.

$$h = \lceil log_2(N+1) \rceil \sim log_2 N$$

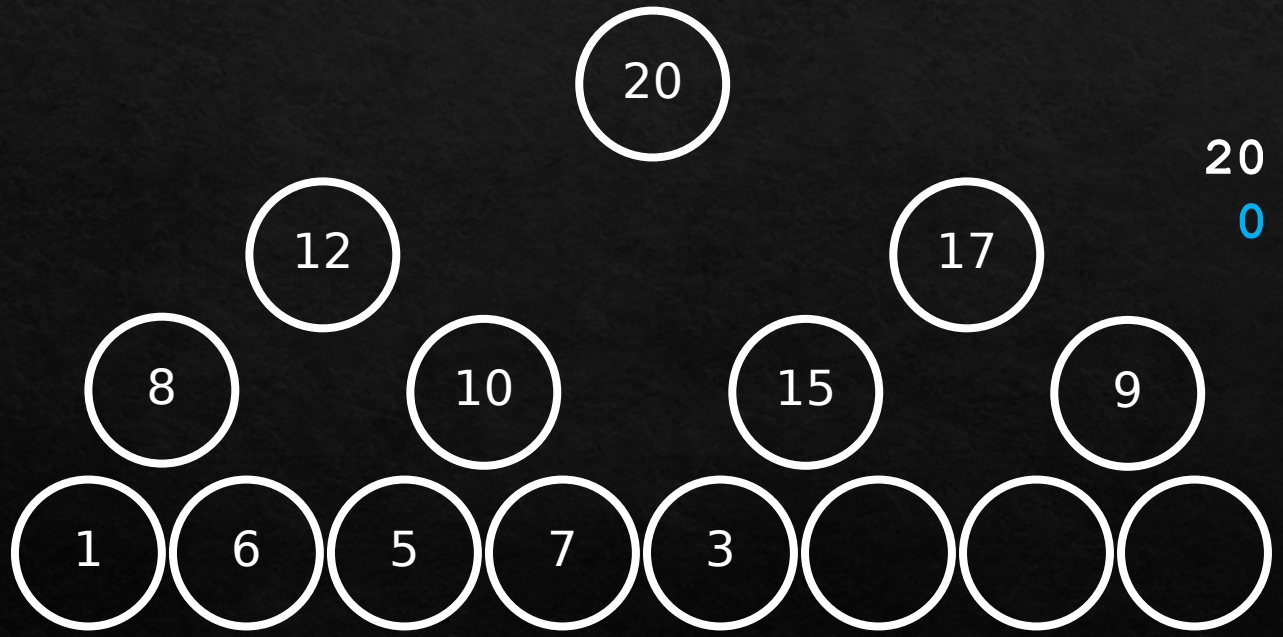
Составление пирамиды из N произвольных элементов занимает в среднем O(N logN).

Реализация пирамиды в массиве

Рассмотрим следующие правила индексирования:

1. вершина пирамиды находится в индексе 0;
2. для элемента с индексом i его потомки имеют индексы 2i + 1 и 2i + 2.

Тогда структуру пирамиды можно выразить в виде массива.



20	12	17	8	10	15	9	1	6	5	7	3
0	1	2	3	4	5	6	7	8	9	10	11

Немного кода

```
void sift_up(int arr[], unsigned last) {
    while (last > 0) {
        auto parent_idx = (last - 1) / 2;
        if (arr[parent_idx] >= arr[last])
            break;
        std::swap(arr[parent_idx], arr[last]);
        last = parent_idx;
    }
}
```

Построение пирамиды

```
for (unsigned next = 0; next != size; ++next) {
    heap[next] = arr[next];
    sift_up(heap, next);
}
```

```
1 0 2 2 3 5 9 8 6 6 1 0 3 5 5 5 7 8 9 0
9 9 5 8 6 3 5 6 8 2 1 0 1 3 5 0 5 2 7 0
```

```
void sift_down(int arr[], unsigned last) {
    unsigned sift_idx = 0, swap_idx;
    unsigned
        childA = sift_idx * 2 + 1,
        childB = sift_idx * 2 + 2;
    while (childB <= last) {
        swap_idx = arr[childA] > arr[childB] ? childA : childB;
        if (arr[sift_idx] >= arr[swap_idx])
            break;

        std::swap(arr[sift_idx], arr[swap_idx]);
        sift_idx = swap_idx;
        childA = sift_idx * 2 + 1,
        childB = sift_idx * 2 + 2;
    }

    if (childA <= last)
        if (arr[sift_idx] < arr[childA])
            std::swap(arr[sift_idx], arr[childA]);
}
```

Пирамидальная сортировка (heap sort)

```
void heap_sort(int arr[], unsigned size) {
    // heapify
    for (unsigned next = 0; next != size; ++next) {
        sift_up(arr, next);
    }

    // sort
    for (unsigned last = size - 1; last != 0; --last) {
        std::swap(arr[last], arr[0]);
        sift_down(arr, last - 1);
    }
}
```

Асимптотическая сложность сортировки составляет O(N logN).

Пирамидальная сортировка неустойчива!

Функции стандартной библиотеки

Heap operations

is_heap (C++11)	checks if the given range is a max heap (function template)
is_heap_until (C++11)	finds the largest subrange that is a max heap (function template)
make_heap	creates a max heap out of a range of elements (function template)
push_heap	adds an element to a max heap (function template)
pop_heap	removes the largest element from a max heap (function template)
sort_heap	turns a max heap into a range of elements sorted in ascending order (function template)

```cpp
#include <algorithm>
int main() {
    unsigned const size = 20;
    int arr[size]
        = {1, 0, 2, 2, 3, 5, 9, 8, 6, 6, 1, 0, 3, 5, 5, 5, 7, 8, 9, 0};

    std::make_heap(arr, arr + size);

    return 0;
}
```

```
1 0 2 2 3 5 9 8 6 6 1 0 3 5 5 5 7 8 9 0
```

```
9 8 9 8 6 5 5 7 6 3 1 0 3 2 5 5 0 2 1 0
```